

PowerRAG 期末汇报融合版演讲稿

这版稿子的讲述逻辑是：先用“鸿雁计划”的立项初心定调，再讲课程阶段 *PowerRAG* 实际做出来的系统、实验和评测闭环。使用方式：按 *PPT* 翻页讲。每页主稿用于背诵，“讲述重点”用于控制现场节奏。

演讲逐字稿与讲述重点 (PPT 翻页对照版)

第 1 页：PowerRAG 各位老师、助教好，我汇报的项目叫 PowerRAG。 这个名字里有两个意思。Power 一方面对应动力装备知识库，另一方面也说明这个系统不是一个普通的问答页面，而是面向动力装备资料的检索增强生成系统。 项目起点可以追溯到之前“鸿雁计划”科研资助申请。当时项目题目是“基于知识图谱增强的工业故障诊断中的检索增强生成 RAG：新框架、新指标与中文评测基准”。这意味着，我们一开始要解决的就不是“把大模型接到网页上”这么简单，而是把真实的工业资料、故障知识、检索方法、图谱关系和评测体系放到同一个工程里。 所以今天我的汇报不会只讲 RAG 是什么，也不会只展示一个网页效果。我想讲清楚的是：我们怎样把零散的 PDF、扫描件、JSON 标注和动力装备资料，逐步变成一个可以检索、可以引用、可以评测、可以继续优化的可信知识系统。	讲述重点 开场先建立项目定位，不急于展开技术细节。
第 2 页：汇报主线 我的汇报分成四条主线。 第一条是资料问答。真实项目里，资料不是干净地躺在数据库里，而是散在 PDF、扫描页、表格、标注文件和维修报告中。系统首先要解决的是资料能不能统一进入。 第二条是关系追踪。动力装备知识不是孤立句子，部件、故障、工况、参数、维修动作之间都有关系。只靠一个文本片段，有时解释不了完整问题。 第三条是综合能力。一个问题如果跨越多个文件、多个部件，系统就需要把相关证据汇总起来，而不是只返回一段最相似文本。 第四条是评测能力。我们不能只说系统“看起来能回答”，还要能用评测题、baseline、失败归因去证明系统到底强在哪里，弱在哪里，下一步该改哪里。 这四条线放在一起，就是我理解的 PowerRAG：它不是一个页面，而是一套从资料进入、证据构建、检索回答到评测优化的完整流程。	讲述重点 先交代四类能力，具体实现放到后续页面。
第 3 页：项目从设想到系统 如果从项目过程来看，它不是一开始就长成现在这样的。 在鸿雁计划申请书里，我们最初设想的是 PowerKB-Agent 三层架构：知识层负责文档预处理和向量库，检索层负责 BM25、向量检索和 reranker，Agent 层负责规划、推理和验证。这个设想完整，但真正落地时，必须先解决更基础的问题。 第一阶段，我们先让资料能够进入系统。最早完成的是 JSON 解析、ChromaDB 写入、查询、统计和导出，目标是先打通最小可运行链路。 第二阶段，真实资料进来之后，问题变了。资料里有扫描型 PDF，有页码，有框选区域，有 OCR，有不同来源的元数据。这时候系统的关键不再是“能不能回答”，而是“资料能不能被正确处理、正确入库、正确保留来源”。 第三阶段，我们开始比较不同检索方式，并加入 GraphRAG 和评测平台。也就是从一个能操作的工作台，逐步走向一个能被评估、能被优化的系统。	讲述重点 突出探索过程，让老师看到项目是被真实资料逐步推动出来的。
第 4 页：我们面对的不是一道问答题 这个项目最核心的起点是：我们面对的不是一道问答题，而是一堆真实资料。 动力装备领域里，资料来源非常复杂。鸿雁计划申请书里也写到，能源动力设备，比如汽轮机、燃气轮机、发电机组，在长期运行和维护中积累了大量技术手册、检修规程、历史故障案例、行业标准和论文资料。这些知识往往散落在纸质文档、电子文档和工程师经验里。 所以如果只是把问题直接丢给大模型，它可能会给出一个看起来合理的回答，但我们很难确认：这个答案到底来自哪份资料？有没有对应页码？有没有漏掉关键故障链条？有没有把不同设备场景混在一起？ 因此，PowerRAG 第一件事不是生成答案，而是把资料变成系统能处理的对象。只有资料先进来，后面的检索、引用、图谱和评测才有意义。	讲述重点 强调“资料复杂”对系统设计的影响，而不是泛泛说资料很多。

第 5 页：初心是把资料变成可信答案

讲述重点

把引用和证据链讲成系统价值，而不是界面上的附加信息。

PowerRAG 的初心，可以概括成一句话：从资料到可信答案。

可信答案不是一句孤立的自然语言，而应该由三部分组成：问题、答案、证据。证据里还要包含来源文件、页码、段落、标题或者原始位置。

举个例子，如果用户问燃气轮机某类故障可能和哪些部件或工况有关，系统不能只回答“可能和压气机、燃烧室或负荷变化有关”。更好的回答应该说明：这个判断来自哪一份维修资料，具体在哪一页，相关段落讲了什么，哪些证据支持这个关系。

这也是我们和普通问答 demo 的区别。普通 demo 更关心回答是否流畅，而 PowerRAG 更关心回答能不能回到资料本身。因为在动力装各场景里，答案如果不能追溯，就很难被检查，也很难被老师、工程师或后续系统继续使用。

第 6 页：第一阶段先让工作台跑起来

讲述重点

承认早期工作台不完整，同时说明它是系统链路的起点。

项目早期，我们做的是一个尚未系统化的工作台。

当时重点不是 GraphRAG，也不是完整评测平台，而是先把最基本的链路跑通：上传资料、解析 JSON、写入 ChromaDB、查看入库状态、做简单查询、统计和导出。

这一阶段基础但必要。它证明了资料不是停留在文件夹里，而是可以进入系统，变成后续检索能够使用的数据。

但问题也很明显。早期工作台更多是“能操作”，还不是“完整系统”。它缺少清晰的数据管线，缺少统一的元数据规范，缺少系统性评测，也没有把失败原因拆出来。所以它只是第一步，不是最终成果。

第 7 页：不同资料必须走不同通道

讲述重点

OCR 属于资料入口工程，不是孤立功能。

真实资料进入系统以后，我们发现不同资料不能用同一种方式处理。

JSON 标注文件结构化程度较高，适合直接解析字段，比如文件名、页码、框选区域、标注内容和类别。文字型 PDF 可以走文本抽取，把页面里的文字直接拿出来。扫描型 PDF 本质上是图片，如果没有 OCR，系统根本看不到里面的文字。

所以我们做了不同的资料入口：JSON 走结构化解析，文字 PDF 走文本抽取，扫描 PDF 先走 OCR，再进入清洗、分块和入库流程。

这里要强调一点：这不是为了堆功能，而是为了让每一种真实资料都能进入同一套后续流程。只有前面的入口统一了，后面的检索、图谱和评测才不会变成空中楼阁。

第 8 页：真实资料没有那么干净

讲述重点

避免展开 OCR 工具细节，重点放在真实资料对数据处理的倒逼。

这一页讲 OCR，也是讲我们为什么会被真实资料倒逼着换方案。

在理想情况下，PDF 里都是干净文本，直接抽取就可以。但实际资料里经常有扫描页、图片页、表格、印章、页眉页脚和不规则排版。对于这些资料，如果没有 OCR，RAG 后面的检索再强也没有用，因为文字根本没有进来。

所以我们尝试了不同 OCR 路线，包括开源方案和云 OCR。不同方案的效果差异显著：有的识别不稳定，有的版面信息丢失，有的对中文和表格更友好。最终要做的不是迷信某个工具，而是根据动力装备资料的真实质量，选择能稳定产出文本的方案。

由此形成一个核心判断：RAG 的上限不只取决于大模型，也取决于数据进入系统之前有没有被处理好。如果资料入口失败，后面的向量库、GraphRAG 和评测都只是形式。

第 9 页：把厚资料变成可检索证据

讲述重点

把 chunk 表述为证据构建，而不是简单切片。

资料进入系统以后，还不能直接拿来回答问题。我们需要把厚资料变成可检索的证据单元，也就是 chunk。

但这里的 chunk 不是简单按字数切开。如果切得太碎，容易丢失上下文；如果切得太长，检索时又很难精准命中。动力装备资料里还有标题、页码、表格、部件名、故障描述和维修步骤，所以分块时必须尽量保留结构。我们在这个阶段关注的是：每个证据块来自哪个文件、哪一页、哪个标题，它周围的上下文是什么，能不能在回答时被引用回来。

这和鸿雁计划里提到的“文档预处理流水线”是对应的。原计划里写的是 PDF、Word 抽取，清洗，语义分块，元数据标注。现在课程阶段，我们做的是这个方向的第一版工程实现。

第 10 页：证据进入检索空间和图谱空间

讲述重点

可简要解释向量, 不展开数学细节; 强调不同空间服务不同问题。

当证据块构建完成以后, 系统才真正开始从“存资料”变成“能找资料”。

这里有几个空间。第一个是关键词检索空间, 比如 BM25 或 keyword, 它对专业术语、型号、部件名、故障名非常敏感。第二个是向量检索空间, 它把文本变成高维语义表示, 用来寻找意思相近的内容。第三个是图谱空间, 它不只保存文本相似度, 而是保存实体之间的关系, 比如部件、故障、工况、参数和维修动作之间的连接。

这些空间不是互相替代的。关键词检索擅长精确术语, 向量检索擅长语义相似, 图谱擅长关系追踪。PowerRAG 要做的是把这些能力组合起来, 根据问题类型选择更合适的路径。

第 11 页：检索不是越复杂越好

讲述重点

数字用于说明评测价值, 不把 keyword 绝对化。

到检索阶段以后, 我们一开始也容易有一个误区: 觉得向量检索、混合检索一定比关键词检索高级。

但我们用 60 道评测题做比较以后, 结果不是这样。当前评测里, keyword 的 question recall@K 是 0.833, hybrid_rrf 是 0.817, dense_hashing 是 0.583。

这个结果说明, 在动力装备资料里, 很多问题本身带有强术语, 比如设备名、故障名、部件名、工况参数。这些词不是普通语义, 而是非常明确的检索锚点。所以关键词检索在这个阶段表现反而很强。

这个结论对我们很重要。它让我们从“凭感觉选技术”转向“用评测判断方法”。复杂方法不是天然更好, 真正关键的是它是否适合当前资料和当前问题。

第 12 页：不同问题应该走不同路线

讲述重点

讲清楚问题路由, 避免听起来只是堆叠检索方式。

既然检索方法各有优势, 系统就不应该无脑走同一条路线。

事实型、定义型、参数型问题, 通常可以优先走标准 RAG 和关键词检索, 因为这类问题往往有明确术语和直接证据。

关系型问题, 比如某个故障和哪些部件有关、某个工况变化会影响哪些指标, 就更适合进入 GraphRAG, 因为它需要沿着实体和关系去追踪。

综合型或全局型问题, 比如比较多个故障模式、总结某类设备维护规律, 就更适合利用社区摘要或全局检索, 把多个证据块、多个节点和多个文档的信息汇总起来。

所以我们现在的理解是: PowerRAG 不应该只是一条固定链路, 而应该是一个问题路由系统。用户输入问题后, 系统先判断问题类型, 再选择 RAG、GraphRAG、社区摘要或全局综合路线。

第 13 页：GraphRAG 把文本连接成实体关系

讲述重点

顺着图讲节点、关系和证据绑定, 不把 GraphRAG 讲成万能方案。

接下来是 GraphRAG。

普通 RAG 擅长在文本片段里找证据, 但动力装备资料里有很多信息不是一句话能讲完的。比如一个故障可能关联多个部件, 一个部件又关联多个工况, 一个维修动作可能对应多个原因和后果。

GraphRAG 的作用, 就是把文本里的设备、部件、故障、工况、参数和维修动作抽取成节点, 再把它们之间的关系抽取成边。这样系统看到的就不只是一堆文本, 而是一张能追踪关系的知识网络。

关键在于: 图谱关系不能脱离原文。每条关系最好都要能回到具体证据, 否则图谱就会变成看起来漂亮但无法验证的图。PowerRAG 里 GraphRAG 的目标, 是让实体关系和原文证据绑定起来。

第 14 页：GraphRAG 不是替代所有 RAG

讲述重点

体现技术边界, 说明 GraphRAG 的适用范围。

GraphRAG 的定位需要保持清晰: 它不是替代普通 RAG 的万能路线, 而是增强关系型问题和全局综合问题。

如果用户问一个明确概念, 比如某个部件的定义、某个步骤的说明, 普通 RAG 可能就已经足够。强行走图谱, 反而会增加复杂度。

但如果问题涉及多个实体之间的关系, 或者需要跨文档综合, GraphRAG 的价值就会出来。比如沿着故障到部件、部件到工况、工况到参数变化这条链路去追踪, 或者把关系密集的一组节点形成社区摘要, 再用于全局回答。

所以我们最终不是在问“到底用 RAG 还是 GraphRAG”, 而是在问“什么问题适合什么路线”。这也是后续评测要继续验证的部分: 同一批问题分别走 local、global 和标准 RAG, 比较它们在准确性、可解释性和引用质量上的差异。

讲述重点

表述为工程化分层已经形成，避免夸大完成度。

随着项目推进，我越来越意识到，页面只是入口，真正重要的是后面的工程分层。

现在 PowerRAG 可以拆成几层来看。

第一层是数据管线，负责上传、解析、OCR、清洗、分块和元数据保留。第二层是存储层，包括 ChromaDB、关键词索引和图谱数据。第三层是检索层，包括 keyword、dense、hybrid 等召回方式。第四层是图谱层，负责实体、关系、社区和证据绑定。第五层是编排层，负责根据问题类型选择路线。第六层是评测层，负责题集、baseline、评分和失败归因。

这个分层其实对应了鸿雁计划里的原始设想：知识层、检索层、Agent 层。不同的是，课程阶段我们没有急着宣称完整 Agent 已经完成，而是先把资料、检索、图谱和评测这条主链路做出来。

讲述重点

把评测讲成“目标、分数、差距、修正”的闭环，不夸成最终成熟的自动评分系统。

评测是这个项目从演示系统走向可信系统的关键步骤。

如果一个 RAG 系统只能演示，就会留下关键缺口：某一次回答看起来不错，但整体效果、失败来源和优化方向都无法被稳定判断。

因此我们做评测平台，不只展示答案，还要检查项目结构、读取评测集、运行 baseline、输出得分、分析失败原因，并把 GraphRAG 覆盖情况和复现质量纳入观察。

这也对应鸿雁计划里原本提到的评测方向。原计划里希望构建 100 到 200 道中文动力装备故障诊断评测题，并使用 HitRate@5、MRR、Faithfulness、Answer Relevancy 等指标。课程阶段我们目前有两条评测线：第一条是面向动力装备语料的 60 题 baseline，用来比较 keyword、dense_hashing 和 hybrid_rrf，判断系统路线；第二条是下载开源 RAG benchmark 后跑的外部评测，用来对照成熟 RAG 项目的质量门槛。

这里有一个能够说明项目过程的节点：我们曾经集中投入数小时，只围绕一个目标做优化，就是让开源 benchmark 的外部质量门禁稳定超过 90 分。对应报告是 external_benchmark_eval_open_source_90_completion_audit_50x2_auto.md，评测集包含 100 个 case，总分达到 93.47 / 100。其中 legal-rag-bench 50 题得分 96.68，ragbench-hotpotqa-validation 50 题得分 90.25。这个结果的意义不是宣称动力装备真实语料已经完成最终工业验收，而是说明我们的评测平台已经能够接入公开 benchmark，并用分数驱动检索、切分、重排和召回策略的调整。

可以放到 PPT 或答辩中的证据表如下：

证据项	阶段记录	说明
动力装备 60 题 baseline	keyword 0.833; hybrid_rrf 0.817; dense_hashing 0.583	用于判断本项目语料上的检索路线，说明强术语场景下 keyword 当前表现更稳
开源 benchmark 外部评测	93.47 / 100	100 个公开 benchmark case，报告为 external_benchmark_eval_open_source_90_completion_audit_50x2_auto.md
legal-rag-bench	50 题，96.68 / 100	法律 RAG 公开 benchmark 子集，外部质量门禁通过
ragbench-hotpotqa-validation	50 题，90.25 / 100	多跳问答公开 benchmark 子集，外部质量门禁通过
Open Source 90 Gate	pass	报告为 open_source_90_external_benchmark_gate_open_source_90_completion_audit_50x2，但它衡量的是 retrieval-first 外部 benchmark，不等于动力装备专家验收

讲述重点

本页可以讲得有力度，因为它体现项目成熟度。

评测的价值不只是给出分数，更重要的是把失败拆解成可执行的工程任务。

在工业资料 RAG 里，失败可能来自很多地方。可能是资料根本没有入库，可能是 OCR 识别错了，可能是 chunk 切得不合适，可能是关键词召回没覆盖，可能是向量模型不理解专业术语，可能是排序把关键证据排到后面，也可能是 GraphRAG 抽取了关系但没有绑定证据。

鸿雁计划里也提到过类似问题，比如因果链条断裂、跨文档推理失败、术语表达不匹配，以及超出知识范围时不能安全拒答。

所以评测之后，我们不应该只说“效果不好”，而要把它转化成下一轮任务：是要补资料，还是修 OCR；是要改分块，还是换 embedding；是要加 reranker，还是重新设计问题路由；是要补 GraphRAG evidence，还是增加全局摘要能力。

这就是完整闭环：资料进入系统，系统回答问题，评测发现失败，失败再反过来指导下一轮优化。外部 benchmark 从低分逐步推到 90 分以上，并在 completion audit 中达到 93.47 分，说明评测不只是展示页面，而是已经开始成为工程优化工具。

第 18 页：现场演示只展示关键入口

讲述重点

现场网络如果不稳定，就讲内置样例和离线结果，避免被接口问题拖住节奏。

如果现场演示，我会尽量只展示两个入口。

第一个入口是资料进入系统。让老师看到不同类型资料不是被简单上传，而是经过解析、OCR、分块、元数据保留和入库。

第二个入口是评测平台。让老师看到系统可以导入项目仓库或加载内置结果，进行结构化评测，输出分数和失败分析。

演示时我不会试图把每个功能都点一遍。因为这个项目最重要的不是页面上按钮多，而是背后的流程完整：资料能进入，证据能找到，答案能引用，失败能归因，下一步能优化。

第 19 页：答辩时要稳住边界

讲述重点

这页用于答辩边界说明，不需要主动展开过多。

如果老师追问项目边界，我会明确说明三点。

第一，PowerRAG 不是要替代大模型，而是给大模型提供可检索、可引用、可评测的领域资料基础。

第二，GraphRAG 不是万能方案，它适合关系追踪、社区摘要和全局综合，但普通事实型问题仍然可以走标准 RAG。

第三，当前课程阶段完成的是第一版工程闭环，包括资料入口、证据构建、检索比较、GraphRAG 方向和评测平台。鸿雁计划里更完整的 Agent、多轮诊断、专家盲评和 200 题以上 benchmark，是后续继续推进的方向。

这种回答既不夸大成果，也能避免项目被理解成单纯页面展示。它说明我们知道系统已经做到哪里，也知道下一步该往哪里走。

第 20 页：最后回到初心

讲述重点

最后总结一下。

PowerRAG 的核心不是“做了一个问答网页”，而是把真实动力装备资料变成一个可信知识系统。

它的主线是：资料先进入系统，经过 OCR、解析、清洗、分块和元数据保留，形成可引用证据；证据进入关键词、向量和图谱空间，支持不同类型问题的检索；GraphRAG 用来增强关系型和全局综合问题；评测平台再用题集、baseline 和失败归因证明系统是否真的有效。

如果对照最初的鸿雁计划，这个课程项目可以看作一个阶段性落地版本。原计划提出的是新框架、新指标和中文评测基准；现在我们已经完成了资料入口、检索实验、GraphRAG 方向和 60 题评测闭环，并且用下载的公开 benchmark 跑出了 100 个 case、93.47 / 100 的外部评测记录。它还不是终点，但已经从一个想法变成了可以继续迭代的工程基础。

下一阶段我会优先推进四件事：第一，更换更正式的 embedding，提高语义召回；第二，引入 reranker，改善召回后的排序；第三，对 GraphRAG 的 local 和 global 问题做同题实测，明确它真正适合哪些问题；第四，扩展评测集和专家评分，让系统从课程项目继续走向可复现实验平台。

我的汇报到这里，谢谢各位老师。

3 分钟压缩版

各位老师好，我汇报的项目叫 PowerRAG。

这个项目的起点不是做一个普通问答网页，而是来自我们之前鸿雁计划里的研究设想：面向动力装备故障诊断，构建知识图谱增强的 RAG 框架、评测指标和中文 benchmark。课程阶段我做的 PowerRAG，可以理解为这个设想的第一版工程化落地。

我们面对的不是一道问答题，而是一批真实资料。动力装备资料里有 PDF、扫描件、JSON 标注、维修报告、页码、表格和图像信息。如果直接交给大模型，它可能说出一段答案，但很难说明答案来自哪里，也很难被检查。

所以项目第一步是资料入口。JSON 走结构化解析，文字 PDF 走文本抽取，扫描 PDF 先走 OCR。这样不同资料才能进入同一套后续流程。

第二步是证据构建。我们不是简单把文本塞进向量库，而是把资料切成带来源、页码、标题和上下文的证据块。这样系统回答时，答案可以回到原始资料，而不是只生成一句看起来合理的话。

第三步是检索实验。我们用 60 道评测题比较了 keyword、hybrid_rrf 和 dense_hashing。结果 keyword 的 question recall@K 是 0.833，hybrid_rrf 是 0.817，dense_hashing 是 0.583。这说明在动力装备资料中，专业术语和型号本身就是强检索信号，复杂方法不一定天然更好，必须用评测决定路线。

第四步是 GraphRAG。普通 RAG 适合回答有直接证据的问题，但故障、部件、工况、参数之间存在关系链，所以我们引入 GraphRAG，把实体和关系组织起来，用于关系追踪、社区摘要和全局综合。但它不是替代所有 RAG，而是根据问题类型进行增强。

最后是评测闭环。系统不能只会演示，还要能评分、能比较 baseline、能分析失败原因。这里要分清两类结果：动力装备 60 题 baseline 用来判断本项目检索路线；公开 benchmark 外部评测则用来对标成熟 RAG 任务。我们下载并接入了 legal-rag-bench 和 ragbench-hotpotqa-validation，各取 50 个 case，最终 completion audit 总分达到 93.47 / 100。失败可能来自 OCR、分块、召回、排序、图谱关系或证据绑定。评测的意义，就是把“效果不好”变成下一轮可以执行的优化任务。

总结来说，PowerRAG 是一个面向动力装备资料的可信问答与评测平台。它从资料进入开始，到证据构建、检索路由、GraphRAG 增强，再到评测归因，形成了一条可以继续迭代的工程链路。下一阶段会继续完善 embedding、reranker、GraphRAG 同题评测和更大规模中文 benchmark。

10 句背诵提纲

1. PowerRAG 不是普通问答网页，而是面向动力装备资料的可信知识系统。
2. 它延续了鸿雁计划里“知识图谱增强工业故障诊断 RAG”的立项初心。
3. 我们面对的是 PDF、扫描件、JSON、维修报告和标准文档，不是一道干净问答题。
4. 第一关是资料入口：JSON 解析，PDF 抽取，扫描件 OCR。
5. 第二关是证据构建：chunk 必须保留来源、页码、标题和上下文。
6. 检索实验说明，动力装备资料里的专业术语让 keyword 在当前 60 题中表现很强。
7. 所以方法不是越复杂越好，而是要用评测决定路线。
8. GraphRAG 的价值在关系追踪、社区摘要和全局综合，不是替代所有

RAG。

9. 评测平台让项目从“能演示”变成“能评分、能归因、能优化”，公开 benchmark 外部评测达到 93.47 / 100。
10. 下一步是更正式的 embedding、reranker、GraphRAG 同题评测和更大规模中文 benchmark。

答辩追问备用回答

如果老师问：你们和普通 RAG 有什么区别？

普通 RAG 更像是把文档切片后做检索问答。我们这里更强调真实资料工程和可信闭环：资料入口要处理 JSON、PDF、扫描件和 OCR；证据块要保留来源和页码；检索方法要用评测比较；GraphRAG 用来补关系型和全局综合问题；最后还要有评测平台做 baseline 和失败归因。所以区别不在于有没有调用大模型，而在于有没有把资料、证据、图谱和评测组织成一个系统。

如果老师问：为什么 keyword 比 dense 还高？

这个结果和领域资料特点有关。动力装备资料里有很多强术语，比如设备名、部件名、故障名、型号和工况参数。这类词本身就是非常明确的检索锚点。当前 dense_hashing 可能没有充分理解这些专业术语，所以表现不如 keyword。这不代表语义检索没有价值，而是说明后续需要更好的 embedding、reranker 和问题路由。

如果老师问：GraphRAG 到底解决什么问题？

GraphRAG 主要解决普通文本片段难以表达的关系型问题。比如故障和部件、工况和参数、维修动作和原因之间的连接。它适合关系追踪、社区摘要和跨文档综合。但对于简单定义或事实问题，普通 RAG 可能更直接，所以我们将 GraphRAG 定位为增强路线，不是替代路线。

如果老师问：评测平台现在成熟吗？

当前是第一版系统化评测闭环，已经可以围绕动力装备 60 题做 baseline 比较和失败分析；同时也能接入下载到本地的公开 RAG benchmark。外部评测里，open_source_90_completion_audit_50x2_auto 这次运行覆盖 100 个 case，总分 93.47 / 100，其中 legal-rag-bench 为 96.68，ragbench-hotpotqa-validation 为 90.25。这里需要说明的是，这代表 retrieval-first 外部 benchmark 质量门禁通过，不等于最终动力装备专家 benchmark 已经完成。按照鸿雁计划，后续目标是扩展到 100 到 200 道以上的中文动力装备故障诊断评测题，并引入更完整的指标和专家评分。

如果老师问：93.47 分能说明什么？

它说明评测系统已经开始发挥工程作用。我们不是凭主观感觉说系统变好了，而是把公开 benchmark 下载到本地，用同一套评测入口反复检查解析、切分、检索、召回和重排策略。最后在 100 个公开 benchmark case 上取得 93.47 / 100，并通过 Open Source 90 External Benchmark Gate。但我不会把它说成最终工业验收结果，因为动力装备真实语料、专家 gold set 和更大规模中文 benchmark 还需要继续补充。

如果老师问：这个项目下一步最重要的是什么？

我认为最重要的是把评测闭环继续做实。具体包括更换正式 embedding，引入 reranker，对 GraphRAG local 和 global 做同题对比，扩展评测集，并把失败归因和任务队列打通。这样项目才能从课程演示继续走向可复现实验平台。